AI ORGANISATION

Pipeline Run Report

IDEA

Build a B2B mobile-first pharmacy ordering app where a retail pharmacist (the retailer) can browse medicines from multiple distributors, place orders, manage a cart, see active and closed orders, and view their store profile. After login, the retailer lands on a 4-tab dashboard: Home (offers carousel + medicine search + quick links to Distributors / Schemes / Generic Medicines / Scan Prescription / Outstanding Payments), Orders (Active / Closed sub-tabs), Cart (line items with distributor + price), Profile (name, license number, favourites, store location).

Decision: WROTE_FILES

TIMESTAMP	2026-05-02T18:59:38.789Z
TOTAL TIME	3 ms across 9 step(s)
VALIDATION	valid · 6 feature(s), 10 task(s)
SELECTED TASK	t-orders-1 (developer, 3h) -> f-orders-tab
QA	PASS
CYBERSECURITY	GO
FILES WRITTEN	1 file(s), 1,796 bytes

This report captures the full agent chain (CEO -> CTO -> Engineering Manager -> Developer -> QA -> Cybersecurity). Each agent's complete output is included in the per-agent sections.

Steps

#	Step	Status	Duration	Notes
1	ceo	ok	0 ms	
2	cto	ok	1 ms	
3	engineering-manager	ok	0 ms	
4	validation	ok	0 ms	6 features, 10 tasks
5	task-selection	ok	0 ms	picked t-orders-1 (developer, 3h) under f-orders-tab
6	developer	ok	1 ms	
7	qa	ok	0 ms	decision=PASS
8	cybersecurity	ok	0 ms	decision=GO
9	file-writer	ok	1 ms	1 file(s), 1796 bytes

Validation

Valid — 6 feature(s), 10 task(s).

Selected task

Task: t-orders-1 — Implement the orders APIs: GET /api/orders?status=active|closed (active = placed | acknowledged | out_for_delivery, closed = delivered | cancelled, sorted newest-first), POST /api/orders (reads the current retailer's cart, fans out into one order per distinct distributorId, copies cart_items into order_items, clears the cart, returns the new orderIds + count). 401 without the session cookie.

Assignee: developer

Estimate: 3 h

Feature: f-orders-tab — Orders tab + place-order API

Files written

1 file(s), 1,796 bytes total.

Path	Bytes
app/api/orders/route.ts	1,796

CEO

Mission: Let an independent retail pharmacist place orders to every medicine distributor they buy from in one mobile-first app, replacing the WhatsApp + phone-call ordering loop they live in today.

OKRS

- Ship a retailer-only build with login and a 4-tab dashboard (Home, Orders, Cart, Profile) that supports the full search -> add-to-cart -> place-order -> see-in-active-orders loop within 14 days
- Convert 50% of the 6 pilot retailers from their current channel to at least one in-app order in the first 30 days post-launch
- Surface an outstanding-payments tile on the Home tab so retailers can see distributor-wise balance without calling, reducing reconciliation calls per retailer by 70% in the same window

PRIORITIES

- Stand up the authenticated 4-tab dashboard shell (mock retailer login + tab navigation) so every later task has a place to land in the UI
- Build the Home tab: search bar at the top, offers carousel, and the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments)
- Build the Cart and Orders tabs end-to-end: cart line items with distributor + total, place-order action, and Active/Closed sub-tabs in Orders
- Build the Profile tab as a read-only retailer card (name, license number, favourites count, store location) plus the outstanding-payments stub it links to from Home

DELEGATION

CTO: Build a mobile-first Next.js 14 (App Router) + Tailwind webapp. Backend is Next.js Route Handlers in the same project; demo data lives in in-memory Maps seeded at boot (one pilot retailer, four distributors, ~20 medicines, three offers). Authentication is a mock cookie-based session: POST /api/auth/login takes a license number and sets a pharmacy-session cookie, middleware guards /dashboard/*. No real distributor integration, no real payments, no Postgres this cycle. Ship by end of day 8 with a runnable end-to-end loop: login -> search a medicine -> add to cart -> place order -> see it under Orders/Active -> view Profile.

CMO: Skip this cycle. Pilot is direct sign-up with the 6 retailers we already have a relationship with; no paid acquisition until cycle 2.

CFO: Back-of-envelope only: distributor commission percent x average order value x monthly orders per retailer. Re-validate before cycle 2; no spend this cycle other than the existing engineering team.

CPO: Own the product copy: empty states (no orders, empty cart, no offers), the 'outstanding payments' tile heading and per-distributor row, the 5 quick-link tile labels and supporting microcopy, and the order-status vocabulary (Placed / Acknowledged / Out for delivery / Delivered / Cancelled). Hand the copy sheet to CTO by end of day 2.

FULL CEO OUTPUT (JSON)


```
{
  "mission": "Let an independent retail pharmacist place orders to every medicine distributor
they buy from in one mobile-first app, replacing the WhatsApp + phone-call ordering loop they
live in today.",
  "okrs": [
    "Ship a retailer-only build with login and a 4-tab dashboard (Home, Orders, Cart, Profile)
that supports the full search -> add-to-cart -> place-order -> see-in-active-orders loop within
14 days",
    "Convert 50% of the 6 pilot retailers from their current channel to at least one in-app order
in the first 30 days post-launch",
```

```
    "Surface an outstanding-payments tile on the Home tab so retailers can see distributor-wise balance without calling, reducing reconciliation calls per retailer by 70% in the same window"
  ],
  "priorities": [
    "Stand up the authenticated 4-tab dashboard shell (mock retailer login + tab navigation) so every later task has a place to land in the UI",
    "Build the Home tab: search bar at the top, offers carousel, and the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments)",
    "Build the Cart and Orders tabs end-to-end: cart line items with distributor + total, place-order action, and Active/Closed sub-tabs in Orders",
    "Build the Profile tab as a read-only retailer card (name, license number, favourites count, store location) plus the outstanding-payments stub it links to from Home"
  ],
  "delegation": {
    "cto": "Build a mobile-first Next.js 14 (App Router) + Tailwind webapp. Backend is Next.js Route Handlers in the same project; demo data lives in in-memory Maps seeded at boot (one pilot retailer, four distributors, ~20 medicines, three offers). Authentication is a mock cookie-based session: POST /api/auth/login takes a license number and sets a pharmacy-session cookie, middleware guards /dashboard/*. No real distributor integration, no real payments, no Postgres this cycle. Ship by end of day 8 with a runnable end-to-end loop: login -> search a medicine -> add to cart -> place order -> see it under Orders/Active -> view Profile.",
    "cmo": "Skip this cycle. Pilot is direct sign-up with the 6 retailers we already have a relationship with; no paid acquisition until cycle 2.",
    "cfo": "Back-of-envelope only: distributor commission percent x average order value x monthly orders per retailer. Re-validate before cycle 2; no spend this cycle other than the existing engineering team.",
    "cpo": "Own the product copy: empty states (no orders, empty cart, no offers), the 'outstanding payments' tile heading and per-distributor row, the 5 quick-link tile labels and supporting microcopy, and the order-status vocabulary (Placed / Acknowledged / Out for delivery / Delivered / Cancelled). Hand the copy sheet to CTO by end of day 2."
  }
}
```

CTO

ARCHITECTURE

Frontend: Next.js 14 (App Router) + Tailwind CSS, mobile-first responsive layout. Authenticated /dashboard route renders a fixed bottom tab bar (Home / Orders / Cart / Profile) on small screens that promotes to a side rail at md+. Client components only where state is needed (search input, cart updates, tab toggles); everything else is a server component reading from the in-process data layer.

Backend: Next.js Route Handlers under app/api/* in the same Next project (no separate Node service). Auth uses a signed cookie set via Next's cookies() API; middleware at the project root guards /dashboard/* and every /api/* route except /api/auth/login. All handlers are typed end-to-end via shared zod schemas in lib/api/contracts.ts.

Database: In-memory storage via lib/db/store.ts (Map<string, Record>), seeded at module load with 1 pilot retailer, 4 distributors, ~20 medicines, 3 offers, and 2 outstanding-payment rows. The schema is shaped to match a future Postgres + Drizzle migration so the same TypeScript types survive the swap; no migration this cycle.

Infrastructure: Single Vercel deploy (frontend + Route Handlers behind one origin). Static assets via Vercel CDN. No external services this cycle. Future cycles: Neon Postgres for persistence, per-distributor HTTP integration via a simple fanout, and S3-compatible storage for prescription scans.

API CONTRACTS (11)

- POST /api/auth/login — Pilot login: takes a retailer license number, looks it up in the in-memory store, sets the pharmacy-session cookie (HttpOnly, SameSite=Lax) and returns the retailer summary. No password this cycle (single pilot retailer per device).
- POST /api/auth/logout — Clears the pharmacy-session cookie.
- GET /api/medicines/search — Case-insensitive substring search over medicine name, brand, and generic name. Empty q returns the full catalogue (used by the search results page when a retailer lands without a query). Each result carries the distributor it ships from so the UI can render the price + supplier chip.
- GET /api/distributors — List all distributors the retailer can buy from. Used by the Home tab quick-link and the cart line items.
- GET /api/offers — List of currently-active offers shown in the Home tab carousel. Returned in sortOrder. Each entry references the distributor running the offer.
- GET /api/cart — Returns the current retailer's cart, grouped by distributor with a per-distributor subtotal and a grand total. Drives the Cart tab.
- POST /api/cart/items — Add a medicine to the current retailer's cart, or increment the qty if it is already there. Validates that the medicineId exists. Returns the updated cart payload (same shape as GET /api/cart) so the Cart tab can render without a second round-trip.
- DELETE /api/cart/items/{medicineId} — Remove a medicine from the current retailer's cart. Returns the updated cart payload.
- POST /api/orders — Place orders from the current retailer's cart. The cart is fanned out into one order per distributor (so the retailer's UI can show each supplier's order separately under Orders > Active). Cart is cleared on success.
- GET /api/orders — List the retailer's orders, filtered by status group. status=active returns placed | acknowledged | out_for_delivery; status=closed returns delivered | cancelled. Sorted newest-first.
- GET /api/profile — Returns the current retailer's profile card plus the outstanding-payments breakdown shown on the Home tab tile.

DATABASE SCHEMA (8)

- **retailers**
  - id: string primary key (uuid in production)
  - name: string not null (display name shown on Profile)
  - owner_name: string not null
  - license_number: string not null unique (used as login id this cycle)
  - store_name: string not null
  - store_address: string not null
  - phone: string not null
  - email: string not null
  - gstin: string not null
  - favourites_medicine_ids: string[] not null default []
  - created_at: timestamp not null
- **distributors**
  - id: string primary key
  - name: string not null
  - region: string not null
  - supplier_code: string not null unique
  - contact_phone: string not null
  - contact_email: string not null
  - rating: number not null (0-5, decimal)
  - created_at: timestamp not null
- **medicines**
  - id: string primary key
  - name: string not null
  - brand: string not null
  - manufacturer: string not null
  - generic_name: string not null
  - distributor_id: string not null references distributors(id)
  - mrp_paise: integer not null (>=0)
  - selling_price_paise: integer not null (>=0)
  - scheme: string nullable (e.g. '10+1 free' or null)
  - pack_size: string not null (e.g. '10 tablets', '100 ml syrup')
  - hsn_code: string not null
  - created_at: timestamp not null
- **offers**
  - id: string primary key
  - title: string not null
  - description: string not null
  - distributor_id: string not null references distributors(id)
  - banner_label: string not null (short text shown on the gradient banner)
  - valid_until: timestamp not null
  - sort_order: integer not null (lower = earlier in the carousel)
  - created_at: timestamp not null
- **cart_items**
  - id: string primary key
  - retailer_id: string not null references retailers(id)
  - medicine_id: string not null references medicines(id)
  - distributor_id: string not null references distributors(id) (denormalised from medicines for easy grouping)
  - qty: integer not null (>=1)
  - unit_price_paise: integer not null (frozen at add-to-cart time)
  - added_at: timestamp not null

    - unique(retailer_id, medicine_id)
- **orders**
    - id: string primary key
    - retailer_id: string not null references retailers(id)
    - distributor_id: string not null references distributors(id)
    - status: string not null (one of: placed | acknowledged | out_for_delivery | delivered | cancelled)
    - placed_at: timestamp not null
    - expected_delivery: timestamp nullable
    - total_paise: integer not null (>=0)
    - item_count: integer not null (>=1)
- **order_items**
    - id: string primary key
    - order_id: string not null references orders(id)
    - medicine_id: string not null references medicines(id)
    - qty: integer not null (>=1)
    - unit_price_paise: integer not null
    - line_total_paise: integer not null
- **outstanding_payments**
    - id: string primary key
    - retailer_id: string not null references retailers(id)
    - distributor_id: string not null references distributors(id)
    - amount_due_paise: integer not null (>=0)
    - last_updated_at: timestamp not null
    - unique(retailer_id, distributor_id)

RISKS (6)

- security: mock cookie auth (single shared session secret, no password, no multi-device invalidation) is intentional for the pilot but unsafe for production; flag for a real auth task in cycle 2
- compliance: pharmacy ordering is regulated (drug schedules, Schedule H1/X tracking, narcotics audit trail). This cycle ignores schedule classification and prescription validation; both are mandatory before any non-pilot rollout
- data loss: in-memory Maps reset on every process restart. Acceptable for a demo, blocking for a customer pilot
- scaling: medicine search is a linear scan over the in-memory store. Fine for ~20 medicines; needs Postgres trigram index or Meilisearch beyond ~1000 SKUs
- mobile UX: 'Scan Prescription' is a placeholder tile this cycle; barcode + OCR scan needs a native shell or WebRTC + a vision model and is out of scope
- outstanding-payments accuracy: the per-distributor balance is currently a static seed; integration with each distributor's ledger is a downstream task and the tile must be labelled as 'last reconciled at <ts>' to avoid retailer confusion

FULL CTO OUTPUT (JSON)


```
{
  "architecture": {
    "frontend": "Next.js 14 (App Router) + Tailwind CSS, mobile-first responsive layout. Authenticated /dashboard route renders a fixed bottom tab bar (Home / Orders / Cart / Profile) on small screens that promotes to a side rail at md+. Client components only where state is needed (search input, cart updates, tab toggles); everything else is a server component reading from the in-process data layer.",
    "backend": "Next.js Route Handlers under app/api/* in the same Next project (no separate Node service). Auth uses a signed cookie set via Next's cookies() API; middleware at the project root guards /dashboard/* and every /api/* route except /api/auth/login. All handlers are typed end-to-end via shared zod schemas in lib/api/contracts.ts.",
    "database": "In-memory storage via lib/db/store.ts (Map<string, Record>), seeded at module load with 1 pilot retailer, 4 distributors, ~20 medicines, 3 offers, and 2 outstanding-payment rows. The schema is shaped to match a future Postgres + Drizzle migration so the same TypeScript types survive the swap; no migration this cycle.",
    "infrastructure": "Single Vercel deploy (frontend + Route Handlers behind one origin). Static assets via Vercel CDN. No external services this cycle. Future cycles: Neon Postgres for
```

```
persistence, per-distributor HTTP integration via a simple fanout, and S3-compatible storage for
prescription scans."
  },
  "apiContracts": [
    {
      "endpoint": "/api/auth/login",
      "method": "POST",
      "description": "Pilot login: takes a retailer license number, looks it up in the in-memory
store, sets the pharmacy-session cookie (HttpOnly, SameSite=Lax) and returns the retailer
summary. No password this cycle (single pilot retailer per device).",
      "request": {
        "licenseNumber": "string"
      },
      "response": {
        "retailer": {
          "id": "string",
          "name": "string",
          "storeName": "string",
          "licenseNumber": "string"
        }
      }
    },
    {
      "endpoint": "/api/auth/logout",
      "method": "POST",
      "description": "Clears the pharmacy-session cookie.",
      "request": {},
      "response": {
        "ok": "true"
      }
    },
    {
      "endpoint": "/api/medicines/search",
      "method": "GET",
      "description": "Case-insensitive substring search over medicine name, brand, and generic
name. Empty q returns the full catalogue (used by the search results page when a retailer lands
without a query). Each result carries the distributor it ships from so the UI can render the
price + supplier chip.",
      "request": {
        "q": "string (optional)"
      },
      "response": {
        "results": [
          {
            "id": "string",
            "name": "string",
            "brand": "string",
            "genericName": "string",
            "distributor": {
              "id": "string",
              "name": "string"
            },
            "mrpPaise": "integer",
            "sellingPricePaise": "integer",
            "scheme": "string | null",
            "packSize": "string"
          }
        ]
      }
    },
    {
      "endpoint": "/api/distributors",
      "method": "GET",
      "description": "List all distributors the retailer can buy from. Used by the Home tab quick-
link and the cart line items.",
      "request": {},
      "response": {
        "distributors": [
          {
            "id": "string",
            "name": "string",
            "region": "string",
            "rating": "number"
          }
        ]
```

```
      }
    },
    {
      "endpoint": "/api/offers",
      "method": "GET",
      "description": "List of currently-active offers shown in the Home tab carousel. Returned in sortOrder. Each entry references the distributor running the offer.",
      "request": {},
      "response": {
        "offers": [
          {
            "id": "string",
            "title": "string",
            "description": "string",
            "distributor": {
              "id": "string",
              "name": "string"
            },
            "validUntil": "ISO-8601 string"
          }
        ]
      }
    },
    {
      "endpoint": "/api/cart",
      "method": "GET",
      "description": "Returns the current retailer's cart, grouped by distributor with a per-distributor subtotal and a grand total. Drives the Cart tab.",
      "request": {},
      "response": {
        "groups": [
          {
            "distributor": {
              "id": "string",
              "name": "string"
            },
            "items": [
              {
                "medicineId": "string",
                "name": "string",
                "brand": "string",
                "qty": "integer",
                "unitPricePaise": "integer",
                "lineTotalPaise": "integer"
              }
            ],
            "subtotalPaise": "integer"
          }
        ],
        "grandTotalPaise": "integer",
        "itemCount": "integer"
      }
    },
    {
      "endpoint": "/api/cart/items",
      "method": "POST",
      "description": "Add a medicine to the current retailer's cart, or increment the qty if it is already there. Validates that the medicineId exists. Returns the updated cart payload (same shape as GET /api/cart) so the Cart tab can render without a second round-trip.",
      "request": {
        "medicineId": "string",
        "qty": "integer (>=1, default 1)"
      },
      "response": {
        "groups": [
          {
            "distributor": {
              "id": "string",
              "name": "string"
            },
            "items": [
              {
                "medicineId": "string",
                "name": "string",
                "brand": "string",
```

```
              "qty": "integer",
              "unitPricePaise": "integer",
              "lineTotalPaise": "integer"
            }
          ],
          "subtotalPaise": "integer"
        }
      ],
      "grandTotalPaise": "integer",
      "itemCount": "integer"
    }
  },
  {
    "endpoint": "/api/cart/items/{medicineId}",
    "method": "DELETE",
    "description": "Remove a medicine from the current retailer's cart. Returns the updated
cart payload.",
    "request": {
      "medicineId": "string (path param)"
    },
    "response": {
      "groups": [
        {
          "distributor": {
            "id": "string",
            "name": "string"
          },
          "items": [
            {
              "medicineId": "string",
              "name": "string",
              "brand": "string",
              "qty": "integer",
              "unitPricePaise": "integer",
              "lineTotalPaise": "integer"
            }
          ],
          "subtotalPaise": "integer"
        }
      ],
      "grandTotalPaise": "integer",
      "itemCount": "integer"
    }
  },
  {
    "endpoint": "/api/orders",
    "method": "POST",
    "description": "Place orders from the current retailer's cart. The cart is fanned out into
one order per distributor (so the retailer's UI can show each supplier's order separately under
Orders > Active). Cart is cleared on success.",
    "request": {},
    "response": {
      "orderIds": [
        "string"
      ],
      "orderCount": "integer"
    }
  },
  {
    "endpoint": "/api/orders",
    "method": "GET",
    "description": "List the retailer's orders, filtered by status group. status=active returns
placed | acknowledged | out_for_delivery; status=closed returns delivered | cancelled. Sorted
newest-first.",
    "request": {
      "status": "active | closed"
    },
    "response": {
      "orders": [
        {
          "id": "string",
          "distributor": {
            "id": "string",
            "name": "string"
          },
```

```
            "status": "placed | acknowledged | out_for_delivery | delivered | cancelled",
            "placedAt": "ISO-8601 string",
            "expectedDelivery": "ISO-8601 string | null",
            "itemCount": "integer",
            "totalPaise": "integer"
          }
        ]
      }
    },
    {
      "endpoint": "/api/profile",
      "method": "GET",
      "description": "Returns the current retailer's profile card plus the outstanding-payments
breakdown shown on the Home tab tile.",
      "request": {},
      "response": {
        "retailer": {
          "id": "string",
          "name": "string",
          "ownerName": "string",
          "licenseNumber": "string",
          "storeName": "string",
          "storeAddress": "string",
          "phone": "string",
          "email": "string",
          "gstin": "string",
          "favouritesCount": "integer"
        },
        "outstanding": {
          "totalPaise": "integer",
          "perDistributor": [
            {
              "distributor": {
                "id": "string",
                "name": "string"
              },
              "amountPaise": "integer"
            }
          ]
        }
      }
    }
  ],
  "databaseSchema": [
    {
      "table": "retailers",
      "fields": [
        "id: string primary key (uuid in production)",
        "name: string not null (display name shown on Profile)",
        "owner_name: string not null",
        "license_number: string not null unique (used as login id this cycle)",
        "store_name: string not null",
        "store_address: string not null",
        "phone: string not null",
        "email: string not null",
        "gstin: string not null",
        "favourites_medicine_ids: string[] not null default []",
        "created_at: timestamp not null"
      ]
    },
    {
      "table": "distributors",
      "fields": [
        "id: string primary key",
        "name: string not null",
        "region: string not null",
        "supplier_code: string not null unique",
        "contact_phone: string not null",
        "contact_email: string not null",
        "rating: number not null (0-5, decimal)",
        "created_at: timestamp not null"
      ]
    },
    {
      "table": "medicines",
```

```
      "fields": [
        "id: string primary key",
        "name: string not null",
        "brand: string not null",
        "manufacturer: string not null",
        "generic_name: string not null",
        "distributor_id: string not null references distributors(id)",
        "mrp_paise: integer not null (>=0)",
        "selling_price_paise: integer not null (>=0)",
        "scheme: string nullable (e.g. '10+1 free' or null)",
        "pack_size: string not null (e.g. '10 tablets', '100 ml syrup')",
        "hsn_code: string not null",
        "created_at: timestamp not null"
      ]
    },
    {
      "table": "offers",
      "fields": [
        "id: string primary key",
        "title: string not null",
        "description: string not null",
        "distributor_id: string not null references distributors(id)",
        "banner_label: string not null (short text shown on the gradient banner)",
        "valid_until: timestamp not null",
        "sort_order: integer not null (lower = earlier in the carousel)",
        "created_at: timestamp not null"
      ]
    },
    {
      "table": "cart_items",
      "fields": [
        "id: string primary key",
        "retailer_id: string not null references retailers(id)",
        "medicine_id: string not null references medicines(id)",
        "distributor_id: string not null references distributors(id) (denormalised from medicines
for easy grouping)",
        "qty: integer not null (>=1)",
        "unit_price_paise: integer not null (frozen at add-to-cart time)",
        "added_at: timestamp not null",
        "unique(retailer_id, medicine_id)"
      ]
    },
    {
      "table": "orders",
      "fields": [
        "id: string primary key",
        "retailer_id: string not null references retailers(id)",
        "distributor_id: string not null references distributors(id)",
        "status: string not null (one of: placed | acknowledged | out_for_delivery | delivered |
cancelled)",
        "placed_at: timestamp not null",
        "expected_delivery: timestamp nullable",
        "total_paise: integer not null (>=0)",
        "item_count: integer not null (>=1)"
      ]
    },
    {
      "table": "order_items",
      "fields": [
        "id: string primary key",
        "order_id: string not null references orders(id)",
        "medicine_id: string not null references medicines(id)",
        "qty: integer not null (>=1)",
        "unit_price_paise: integer not null",
        "line_total_paise: integer not null"
      ]
    },
    {
      "table": "outstanding_payments",
      "fields": [
        "id: string primary key",
        "retailer_id: string not null references retailers(id)",
        "distributor_id: string not null references distributors(id)",
        "amount_due_paise: integer not null (>=0)",
        "last_updated_at: timestamp not null",
```

```
        "unique(retailer_id, distributor_id)"
      ]
    }
  ],
  "risks": [
    "security: mock cookie auth (single shared session secret, no password, no multi-device invalidation) is intentional for the pilot but unsafe for production; flag for a real auth task in cycle 2",
    "compliance: pharmacy ordering is regulated (drug schedules, Schedule H1/X tracking, narcotics audit trail). This cycle ignores schedule classification and prescription validation; both are mandatory before any non-pilot rollout",
    "data loss: in-memory Maps reset on every process restart. Acceptable for a demo, blocking for a customer pilot",
    "scaling: medicine search is a linear scan over the in-memory store. Fine for ~20 medicines; needs Postgres trigram index or Meilisearch beyond ~1000 SKUs",
    "mobile UX: 'Scan Prescription' is a placeholder tile this cycle; barcode + OCR scan needs a native shell or WebRTC + a vision model and is out of scope",
    "outstanding-payments accuracy: the per-distributor balance is currently a static seed; integration with each distributor's ledger is a downstream task and the tile must be labelled as 'last reconciled at <ts>' to avoid retailer confusion"
  ]
}
```

Engineering Manager

FEATURES (6)

- f-shell-auth (high) — App shell + mock retailer auth
- f-data-spine (high) — In-memory data spine + shared types
- f-home-tab (high) — Home tab
- f-orders-tab (high) — Orders tab + place-order API
- f-cart-tab (high) — Cart tab + cart APIs
- f-profile-tab (medium) — Profile tab

TASKS (10)

- t-orders-1 -> f-orders-tab · developer · 3h - Implement the orders APIs: GET /api/orders?status=active|closed (active = placed | acknowledged | out_for_delivery, closed = delivered | cancelled, sorted newest-first), POST /api/orders (reads the current retailer's cart, fans out into one order per distinct distributorId, copies cart_items into order_items, clears the cart, returns the new orderIds + count). 401 without the session cookie.
- t-shell-1 -> f-shell-auth · developer · 4h - Set up the Next.js 14 + Tailwind project skeleton: package.json (next 14.2, react 18, tailwind 3, typescript 5, vitest), tsconfig.json with the @/* alias, next.config.mjs, postcss.config.mjs, tailwind.config.ts (mobile-first content globs), app/globals.css with the Tailwind directives + base background, app/layout.tsx with the html/body shell, and the public login page at app/page.tsx that posts to /api/auth/login. No auth handler yet (lands in t-shell-2); the page just calls fetch and redirects on success.
- t-shell-2 -> f-shell-auth · developer · 4h - Implement the mock auth: POST /api/auth/login (look up the retailer by license number against the in-memory store, set the pharmacy-session cookie HttpOnly+SameSite=Lax with a 30-day max-age), POST /api/auth/logout (clear the cookie), middleware.ts that gates /dashboard/* and every /api/* route except /api/auth/login (returns 401 JSON if missing cookie), a small lib/auth/session.ts helper that reads the current retailerId from the request cookies, and the authenticated app/dashboard/layout.tsx that renders the 4-tab bottom navigation (Home / Orders / Cart / Profile) and a stub app/dashboard/page.tsx that redirects to /dashboard/home.
- t-data-1 -> f-data-spine · developer · 4h - Implement the in-memory data spine: lib/types.ts (TypeScript types for Retailer, Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment, OrderStatus, derived Money helpers) and lib/db/store.ts (Map-backed stores seeded at module import with 1 pilot retailer, 4 distributors, 20 medicines spanning the four distributors, 3 active offers, 2 outstanding-payment rows; CRUD helpers - getRetailer, listDistributors, listOffers, searchMedicines(q), getMedicine(id), getCart(retailerId), addCartItem, removeCartItem, clearCart, listOrders(retailerId, statusGroup), createOrdersFromCart(retailerId), getOutstandingForRetailer(retailerId)).
- t-home-1 -> f-home-tab · developer · 3h - Implement the read APIs that drive the Home tab: GET /api/medicines/search (case-insensitive substring over name + brand + generic, returns each result with its distributor), GET /api/distributors (id/name/region/rating list), GET /api/offers (active offers in sortOrder, each with its distributor). All three use the auth-guarded session helper to identify the retailer (search results may carry per-retailer favourites later) and return 401 if the cookie is missing.
- t-home-2 -> f-home-tab · developer · 4h - Build the Home tab page at app/dashboard/home/page.tsx: greeting line with the retailer name, a search bar at the top that links to /dashboard/search?q=..., a horizontally scrollable offers carousel reading from GET /api/offers, the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and an outstanding-payments summary tile that calls GET /api/profile and shows the total + per-distributor breakdown. Mobile-first Tailwind layout, snap-scroll on the carousel.
- t-cart-1 -> f-cart-tab · developer · 3h - Implement the cart APIs: GET /api/cart (returns the cart grouped by distributor with subtotals + grand total + itemCount, derived from the in-memory store), POST /api/cart/items (validates medicineId exists, increments qty if already present, returns the

same grouped payload), DELETE /api/cart/items/[medicineId] (removes the line, returns the same grouped payload). All three resolve the retailer via the session helper and 401 without it.

- t-cart-2 -> f-cart-tab · developer · 3h - Build the Cart tab page at app/dashboard/cart/page.tsx: groups the cart by distributor (heading per supplier with subtotal), each line shows name + brand + qty stepper + remove button + line total, sticky footer with the grand total and a Place Order button that POSTs to /api/orders and on success routes to /dashboard/orders?status=active. Empty-state copy when the cart is empty.
- t-orders-2 -> f-orders-tab · developer · 3h - Build the Orders tab page at app/dashboard/orders/page.tsx with Active and Closed sub-tabs (segmented control). Each sub-tab calls GET /api/orders with the matching status group and renders order cards (distributor name, status pill with status-color, item count, total, placed-at relative time, expected-delivery when present). Empty-state copy for both tabs.
- t-profile-1 -> f-profile-tab · developer · 3h - Implement GET /api/profile (returns the current retailer plus the outstanding-payments breakdown grouped by distributor) and build the Profile tab page at app/dashboard/profile/page.tsx that renders the retailer card (avatar initials, name, store name, license number, owner name, GSTIN, store address, phone, email, favourites count) and an outstanding-payments section listing each distributor's amount-due with the grand total. A logout button at the bottom POSTs /api/auth/logout and routes to /.

FULL ENGINEERING MANAGER OUTPUT (JSON)


```
{
  "features": [
    {
      "id": "f-shell-auth",
      "name": "App shell + mock retailer auth",
      "description": "Next.js 14 + Tailwind project skeleton, the login page, and the
authenticated /dashboard layout with the bottom tab bar that every other feature lives inside.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-data-spine",
      "name": "In-memory data spine + shared types",
      "description": "Single source of truth for retailers, distributors, medicines, offers, cart
items, orders, order items, and outstanding payments. In-memory Maps with seed data, plus the
shared TypeScript types every Route Handler and page imports.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-home-tab",
      "name": "Home tab",
      "description": "Search bar at the top, offers carousel, the 5 quick-link tiles
(Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and the
supporting catalogue / offers / distributors APIs.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-orders-tab",
      "name": "Orders tab + place-order API",
      "description": "Orders list page with Active and Closed sub-tabs, plus the POST /api/orders
fanout (cart -> one order per distributor) and GET /api/orders status-grouped reader.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-cart-tab",
      "name": "Cart tab + cart APIs",
      "description": "Cart page grouped by distributor with per-distributor subtotal and a grand
total, plus the GET / POST / DELETE cart endpoints that drive it.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-profile-tab",
      "name": "Profile tab",
```

```
      "description": "Read-only retailer card (name, owner, license number, store address, GSTIN, phone, email, favourites count) plus the GET /api/profile endpoint that returns the retailer plus the outstanding-payments breakdown.",
      "priority": "medium",
      "status": "pending"
    }
  ],
  "tasks": [
    {
      "id": "t-orders-1",
      "featureId": "f-orders-tab",
      "description": "Implement the orders APIs: GET /api/orders?status=active|closed (active = placed | acknowledged | out_for_delivery, closed = delivered | cancelled, sorted newest-first), POST /api/orders (reads the current retailer's cart, fans out into one order per distinct distributorId, copies cart_items into order_items, clears the cart, returns the new orderIds + count). 401 without the session cookie.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-shell-1",
      "featureId": "f-shell-auth",
      "description": "Set up the Next.js 14 + Tailwind project skeleton: package.json (next 14.2, react 18, tailwind 3, typescript 5, vitest), tsconfig.json with the @/* alias, next.config.mjs, postcss.config.mjs, tailwind.config.ts (mobile-first content globs), app/globals.css with the Tailwind directives + base background, app/layout.tsx with the html/body shell, and the public login page at app/page.tsx that posts to /api/auth/login. No auth handler yet (lands in t-shell-2); the page just calls fetch and redirects on success.",
      "assignedTo": "developer",
      "estimatedHours": 4,
      "status": "pending"
    },
    {
      "id": "t-shell-2",
      "featureId": "f-shell-auth",
      "description": "Implement the mock auth: POST /api/auth/login (look up the retailer by license number against the in-memory store, set the pharmacy-session cookie HttpOnly+SameSite=Lax with a 30-day max-age), POST /api/auth/logout (clear the cookie), middleware.ts that gates /dashboard/* and every /api/* route except /api/auth/login (returns 401 JSON if missing cookie), a small lib/auth/session.ts helper that reads the current retailerId from the request cookies, and the authenticated app/dashboard/layout.tsx that renders the 4-tab bottom navigation (Home / Orders / Cart / Profile) and a stub app/dashboard/page.tsx that redirects to /dashboard/home.",
      "assignedTo": "developer",
      "estimatedHours": 4,
      "status": "pending"
    },
    {
      "id": "t-data-1",
      "featureId": "f-data-spine",
      "description": "Implement the in-memory data spine: lib/types.ts (TypeScript types for Retailer, Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment, OrderStatus, derived Money helpers) and lib/db/store.ts (Map-backed stores seeded at module import with 1 pilot retailer, 4 distributors, 20 medicines spanning the four distributors, 3 active offers, 2 outstanding-payment rows; CRUD helpers - getRetailer, listDistributors, listOffers, searchMedicines(q), getMedicine(id), getCart(retailerId), addCartItem, removeCartItem, clearCart, listOrders(retailerId, statusGroup), createOrdersFromCart(retailerId), getOutstandingForRetailer(retailerId)).",
      "assignedTo": "developer",
      "estimatedHours": 4,
      "status": "pending"
    },
    {
      "id": "t-home-1",
      "featureId": "f-home-tab",
      "description": "Implement the read APIs that drive the Home tab: GET /api/medicines/search (case-insensitive substring over name + brand + generic, returns each result with its distributor), GET /api/distributors (id/name/region/rating list), GET /api/offers (active offers in sortOrder, each with its distributor). All three use the auth-guarded session helper to identify the retailer (search results may carry per-retailer favourites later) and return 401 if the cookie is missing.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
```

```
    {
      "id": "t-home-2",
      "featureId": "f-home-tab",
      "description": "Build the Home tab page at app/dashboard/home/page.tsx: greeting line with the retailer name, a search bar at the top that links to /dashboard/search?q=..., a horizontally scrollable offers carousel reading from GET /api/offers, the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and an outstanding-payments summary tile that calls GET /api/profile and shows the total + per-distributor breakdown. Mobile-first Tailwind layout, snap-scroll on the carousel.",
      "assignedTo": "developer",
      "estimatedHours": 4,
      "status": "pending"
    },
    {
      "id": "t-cart-1",
      "featureId": "f-cart-tab",
      "description": "Implement the cart APIs: GET /api/cart (returns the cart grouped by distributor with subtotals + grand total + itemCount, derived from the in-memory store), POST /api/cart/items (validates medicineId exists, increments qty if already present, returns the same grouped payload), DELETE /api/cart/items/[medicineId] (removes the line, returns the same grouped payload). All three resolve the retailer via the session helper and 401 without it.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-cart-2",
      "featureId": "f-cart-tab",
      "description": "Build the Cart tab page at app/dashboard/cart/page.tsx: groups the cart by distributor (heading per supplier with subtotal), each line shows name + brand + qty stepper + remove button + line total, sticky footer with the grand total and a Place Order button that POSTs to /api/orders and on success routes to /dashboard/orders?status=active. Empty-state copy when the cart is empty.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-orders-2",
      "featureId": "f-orders-tab",
      "description": "Build the Orders tab page at app/dashboard/orders/page.tsx with Active and Closed sub-tabs (segmented control). Each sub-tab calls GET /api/orders with the matching status group and renders order cards (distributor name, status pill with status-color, item count, total, placed-at relative time, expected-delivery when present). Empty-state copy for both tabs.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    },
    {
      "id": "t-profile-1",
      "featureId": "f-profile-tab",
      "description": "Implement GET /api/profile (returns the current retailer plus the outstanding-payments breakdown grouped by distributor) and build the Profile tab page at app/dashboard/profile/page.tsx that renders the retailer card (avatar initials, name, store name, license number, owner name, GSTIN, store address, phone, email, favourites count) and an outstanding-payments section listing each distributor's amount-due with the grand total. A logout button at the bottom POSTs /api/auth/logout and routes to /.",
      "assignedTo": "developer",
      "estimatedHours": 3,
      "status": "pending"
    }
  ]
}
```

Developer

Task: t-orders-1

IMPLEMENTATION PLAN

Land both order endpoints in a single route file (Next 14 supports multiple HTTP methods per route.ts). GET /api/orders reads ?status (default 'active', validates against the union 'active' | 'closed' and returns 400 on anything else), then calls listOrders(retailerId, group) - which already filters by ACTIVE_STATUSES vs CLOSED_STATUSES per t-data-1 - and maps each row into the documented UI projection (id, status, placedAt as ISO string, expectedDelivery as ISO string or null, itemCount, totalPaise, distributor: { id, name }). POST /api/orders calls createOrdersFromCart(retailerId) - which fans out one Order per distinct distributorId, copies CartItems into OrderItems, and clears the cart - and returns { orderIds, orderCount } per the CTO contract. POST returns 409 'cart is empty' when there is nothing to fan out so the Cart UI can surface a meaningful message instead of the generic '0 orders created'. Both handlers short-circuit to 401 without the session cookie and set dynamic = 'force-dynamic'.

FILES PRODUCED (1)

- `app/api/orders/route.ts (1,796 bytes)`

TESTS (2)

- GET /api/orders validates ?status and returns the documented projection.
- POST /api/orders fans out an empty/non-empty cart correctly.

NOTES (3)

- GET defaults to ?status=active because that's where the retailer's eyes go first when they open the Orders tab. Anything other than 'active' or 'closed' is a 400 with a strict error message - documenting the contract in the response body so a client doesn't have to guess.
- POST returns 409 (Conflict) instead of 200 with orderCount: 0 when the cart is empty. 409 is the correct semantic for 'the resource is in a state that cannot satisfy this request'; it lets the Cart UI render an empty-state message without misreading a 200 as success.
- Both endpoints live in the same route.ts because Next 14 supports multiple HTTP methods per route file. This keeps the orders surface in one place and avoids a needless second file.

FULL DEVELOPER OUTPUT (JSON)


```
{
  "taskId": "t-orders-1",
  "implementationPlan": "Land both order endpoints in a single route file (Next 14 supports
multiple HTTP methods per route.ts). GET /api/orders reads ?status (default 'active', validates
against the union 'active' | 'closed' and returns 400 on anything else), then calls
listOrders(retailerId, group) - which already filters by ACTIVE_STATUSES vs CLOSED_STATUSES per t-
data-1 - and maps each row into the documented UI projection (id, status, placedAt as ISO string,
expectedDelivery as ISO string or null, itemCount, totalPaise, distributor: { id, name }). POST /
api/orders calls createOrdersFromCart(retailerId) - which fans out one Order per distinct
distributorId, copies CartItems into OrderItems, and clears the cart - and returns { orderIds,
orderCount } per the CTO contract. POST returns 409 'cart is empty' when there is nothing to fan
out so the Cart UI can surface a meaningful message instead of the generic '0 orders created'.
Both handlers short-circuit to 401 without the session cookie and set dynamic = 'force-dynamic'.",
  "files": [
    {
      "path": "app/api/orders/route.ts",
      "content": "import { NextResponse } from \"next/server\";\n\nimport { getSession } from \"@/
lib/auth/session\";\nimport {\n  createOrdersFromCart,\n  getDistributor,\n  listOrders,\n} from
\"@/lib/db/store\";\nimport { type OrderStatusGroup } from \"@/lib/types\";\n\nexport const
dynamic = \"force-dynamic\";\n\nexport async function GET(request: Request): Promise<Response>
{\n  const session = getSession();\n  if (!session) {\n    return NextResponse.json({ error:
\"unauthenticated\" }, { status: 401 });\n  }\n  const { searchParams } = new URL(request.url);
\n  const raw = (searchParams.get(\"status\") ?? \"active\").toLowerCase();\n  if (raw !==
\"active\" && raw !== \"closed\") {\n    return NextResponse.json(\n      { error: \"status must
be 'active' or 'closed'\" },\n      { status: 400 },\n    );\n  }\n  const group = raw as
OrderStatusGroup;\n  const orders = listOrders(session.retailerId, group).map((o) => {\n    const
```

```
d = getDistributor(o.distributorId);\n    return {\n      id: o.id,\n      status: o.status,\n      placedAt: o.placedAt.toISOString(),\n      expectedDelivery: o.expectedDelivery ? o.expectedDelivery.toISOString() : null,\n      itemCount: o.itemCount,\n      totalPaise: o.totalPaise,\n      distributor: d\n        ? { id: d.id, name: d.name }\n        : { id: o.distributorId, name: \"Unknown distributor\" },\n    };\n  });\n  return NextResponse.json({ orders });\n}\n\nexport async function POST(): Promise<Response> {\n  const session = getSession();\n  if (!session) {\n    return NextResponse.json({ error: \"unauthenticated\" }, { status: 401 });\n  }\n  const created = createOrdersFromCart(session.retailerId);\n  if (created.length === 0) {\n    return NextResponse.json({ error: \"cart is empty\" }, { status: 409 });\n  }\n  return NextResponse.json({\n    orderIds: created.map((c) => c.id),\n    orderCount: created.length,\n  });\n}\n"
    }
  ],
  "tests": [
    {
      "description": "GET /api/orders validates ?status and returns the documented projection.",
      "code": "import { describe, expect, it, vi } from \"vitest\";\n\nlet hasCookie = true;\nvi.mock(\"next/headers\", () => ({\n  cookies: () => ({ get: () => (hasCookie ? { value: \"r-pilot-1\" } : undefined) }),\n}));\n\nimport { GET } from \"./route\";\n\ninterface OrderRow {\n  id: string;\n  status: string;\n  placedAt: string;\n  expectedDelivery: string | null;\n  itemCount: number;\n  totalPaise: number;\n  distributor: { id: string; name: string };\n}\n\ndescribe(\"GET /api/orders\", () => {\n  it(\"defaults to active when ?status is missing\", async () => {\n    const res = await GET(new Request(\"http://localhost/api/orders\"));\n    expect(res.status).toBe(200);\n    const body = (await res.json()) as { orders: OrderRow[] };\n    for (const o of body.orders) {\n      expect([\"placed\", \"acknowledged\", \"out_for_delivery\"]).toContain(o.status);\n    }\n  });\n\n  it(\"returns 400 on invalid status\", async () => {\n    const res = await GET(new Request(\"http://localhost/api/orders?status=foo\"));\n    expect(res.status).toBe(400);\n  });\n\n  it(\"returns 401 without a cookie\", async () => {\n    hasCookie = false;\n    const res = await GET(new Request(\"http://localhost/api/orders\"));\n    expect(res.status).toBe(401);\n    hasCookie = true;\n  });\n\n  it(\"closed status returns delivered/cancelled orders\", async () => {\n    const res = await GET(new Request(\"http://localhost/api/orders?status=closed\"));\n    const body = (await res.json()) as { orders: OrderRow[] };\n    for (const o of body.orders) {\n      expect([\"delivered\", \"cancelled\"]).toContain(o.status);\n    }\n  });\n\n  it(\"each row carries the documented fields including distributor sub-object\", async () => {\n    const res = await GET(new Request(\"http://localhost/api/orders?status=active\"));\n    const body = (await res.json()) as { orders: OrderRow[] };\n    expect(body.orders.length).toBeGreaterThan(0);\n    const o = body.orders[0];\n    expect(o).toHaveProperty(\"distributor.id\");\n    expect(o).toHaveProperty(\"distributor.name\");\n    expect(typeof o.placedAt).toBe(\"string\");\n  });\n});\n"
    },
    {
      "description": "POST /api/orders fans out an empty/non-empty cart correctly.",
      "code": "import { describe, expect, it, vi } from \"vitest\";\n\nvi.mock(\"next/headers\", () => ({\n  cookies: () => ({ get: () => ({ value: \"r-pilot-1\" }) }),\n}));\n\nimport { POST } from \"./route\";\nimport { addCartItem, clearCart } from \"@/lib/db/store\";\n\ndescribe(\"POST /api/orders\", () => {\n  beforeEach(() => clearCart(\"r-pilot-1\"));\n\n  it(\"returns 409 when the cart is empty\", async () => {\n    const res = await POST();\n    expect(res.status).toBe(409);\n  });\n\n  it(\"fans out one order per distinct distributor and returns orderIds\", async () => {\n    addCartItem(\"r-pilot-1\", \"m-paracetamol-500\", 1); // d-mediplus\n    addCartItem(\"r-pilot-1\", \"m-vitd3-60k\", 1); // d-omsai\n    const res = await POST();\n    expect(res.status).toBe(200);\n    const body = (await res.json()) as { orderIds: string[]; orderCount: number };\n    expect(body.orderCount).toBe(2);\n    expect(body.orderIds).toHaveLength(2);\n  });\n});\n"
    }
  ],
  "notes": [
    "GET defaults to ?status=active because that's where the retailer's eyes go first when they open the Orders tab. Anything other than 'active' or 'closed' is a 400 with a strict error message - documenting the contract in the response body so a client doesn't have to guess.",
    "POST returns 409 (Conflict) instead of 200 with orderCount: 0 when the cart is empty. 409 is the correct semantic for 'the resource is in a state that cannot satisfy this request'; it lets the Cart UI render an empty-state message without misreading a 200 as success.",
    "Both endpoints live in the same route.ts because Next 14 supports multiple HTTP methods per route file. This keeps the orders surface in one place and avoids a needless second file."
  ]
}
```

QA

Decision:	**PASS**
Test plan:	11 item(s)
Results:	11 pass, 0 fail
Bugs:	0

BUGS (0)

(none)

FULL QA OUTPUT (JSON)


```
{
  "taskId": "t-orders-1",
  "testPlan": [
    "Verify the file set lands both order endpoints in a single route file: app/api/orders/
route.ts (exports GET + POST).",
    "Verify GET /api/orders defaults to ?status=active when the parameter is missing.",
    "Verify GET /api/orders returns 400 'status must be active or closed' on invalid status
(e.g. ?status=foo, ?status=delivered).",
    "Verify GET /api/orders returns 401 without a session cookie.",
    "Verify GET /api/orders maps each row to the documented projection: id, status, placedAt (ISO
string), expectedDelivery (ISO string or null), itemCount, totalPaise, distributor: { id,
name }.",
    "Verify GET /api/orders 'closed' returns only delivered/cancelled rows; 'active' returns only
placed/acknowledged/out_for_delivery rows.",
    "Verify POST /api/orders returns 409 'cart is empty' when nothing to fan out.",
    "Verify POST /api/orders fans out one order per distinct distributor and returns { orderIds,
orderCount } per the CTO contract.",
    "Verify POST /api/orders clears the cart on success (downstream invariant; relies on
createOrdersFromCart).",
    "Verify dynamic = 'force-dynamic' is exported.",
    "Search files[].content for forbidden placeholder strings."
  ],
  "results": [
    {
      "test": "Single route file exports GET + POST.",
      "status": "pass",
      "details": "app/api/orders/route.ts declares `export async function GET` and `export async
function POST`. Next 14 multi-method route convention."
    },
    {
      "test": "GET defaults ?status to active.",
      "status": "pass",
      "details": "raw = (searchParams.get('status') ?? 'active').toLowerCase(). When the param is
missing, falls into the active branch and listOrders is called with group='active'. Confirmed by
tracing GET head."
    },
    {
      "test": "GET returns 400 on invalid status.",
      "status": "pass",
      "details": "if (raw !== 'active' && raw !== 'closed') -> 400 'status must be active or
closed'. ?status=delivered (a valid OrderStatus but not a valid GROUP) is correctly rejected
because the parameter is the *group*, not the status itself."
    },
    {
      "test": "GET returns 401 without cookie.",
      "status": "pass",
      "details": "if (!session) -> 401 'unauthenticated'. Same defence-in-depth pattern as the
other authenticated reads."
    },
    {
      "test": "GET projection matches the CTO contract.",
      "status": "pass",
      "details": "Each row mapped to { id, status, placedAt: o.placedAt.toISOString(),
expectedDelivery: o.expectedDelivery ? toISOString : null, itemCount, totalPaise, distributor:
{ id, name } | { id, name: 'Unknown distributor' } }. ISO conversion explicit because Date does
not auto-serialise as ISO via NextResponse.json's default."
    },
    {
      "test": "Status grouping is correct.",
```

```
      "status": "pass",
      "details": "listOrders is the source of truth for filtering (verified in t-data-1 audit). The route only translates the group string and trusts the helper. Co-located vitest verifies that for both 'active' and 'closed' every returned row has a status from the corresponding ACTIVE/CLOSED set."
    },
    {
      "test": "POST 409 on empty cart.",
      "status": "pass",
      "details": "createOrdersFromCart returns []. if (created.length === 0) -> 409 'cart is empty'. Cart UI can show an empty-state message rather than misreading a 200 as success."
    },
    {
      "test": "POST fan-out + response shape.",
      "status": "pass",
      "details": "On non-empty cart: createOrdersFromCart returns Array<{ id, distributorId }>. Response: { orderIds: created.map(c => c.id), orderCount: created.length }. Co-located vitest verifies a 2-distributor cart produces orderCount: 2."
    },
    {
      "test": "POST clears the cart on success.",
      "status": "pass",
      "details": "createOrdersFromCart calls clearCart(retailerId) at the end of its synchronous block (verified in t-data-1 audit). The route does not need to clear it again. Atomicity is guaranteed by the synchronous JS event loop in the in-memory implementation."
    },
    {
      "test": "Force-dynamic exported.",
      "status": "pass",
      "details": "`export const dynamic = 'force-dynamic'` declared after imports. Required because both GET and POST call cookies() via getSession()."
    },
    {
      "test": "No forbidden placeholder strings.",
      "status": "pass",
      "details": "Scanned. Zero matches."
    }
  ],
  "bugs": [],
  "decision": "PASS"
}
```

Cybersecurity

Decision: GO

Summary: Audited the orders endpoint that exports both GET and POST. Both handlers resolve the retailer via the session cookie (never from the body), the ?status query parameter is validated against a closed enum, the POST has no body to validate (it operates on the server-side cart), and the response shapes are typed JSON projections of in-memory rows. No critical or high-severity vulnerabilities were found. One medium-severity item (no rate limit on POST) is recorded; the same risk profile as t-cart-1 / t-shell-2 endpoints.

Prompt-injection risk: None. Neither handler issues an LLM call, performs LLM-mediated I/O, renders untrusted text, or executes shell or filesystem operations. The status query parameter is validated against a closed enum before reaching any logic; the POST has no body. All response data flows from typed in-memory rows through NextResponse.json's automatic escaping.

VULNERABILITIES (1)

MEDIUM

POST /api/orders has no rate limit. An authenticated retailer (or anyone holding the unsigned cookie from t-shell-2) can spam the endpoint to grow the orders Map unboundedly. Each call requires a non-empty cart, but a paired loop of POST /api/cart/items + POST /api/orders + repeat would memory-exhaust a Vercel function instance.

- Recommendation: Cycle 2: pair the order POST with the same per-session rate limiter recommended for the cart endpoints (e.g. 10 orders / minute). Belt-and-braces option: enforce a max-orders-per-day-per-retailer cap (e.g. 200) at createOrdersFromCart and return 429 when exceeded.

REQUIRED FIXES (0)

(none)

FULL CYBERSECURITY OUTPUT (JSON)


```
{
  "taskId": "t-orders-1",
  "summary": "Audited the orders endpoint that exports both GET and POST. Both handlers resolve the retailer via the session cookie (never from the body), the ?status query parameter is validated against a closed enum, the POST has no body to validate (it operates on the server-side cart), and the response shapes are typed JSON projections of in-memory rows. No critical or high-severity vulnerabilities were found. One medium-severity item (no rate limit on POST) is recorded; the same risk profile as t-cart-1 / t-shell-2 endpoints.",
  "vulnerabilities": [
    {
      "severity": "medium",
      "description": "POST /api/orders has no rate limit. An authenticated retailer (or anyone holding the unsigned cookie from t-shell-2) can spam the endpoint to grow the orders Map unboundedly. Each call requires a non-empty cart, but a paired loop of POST /api/cart/items + POST /api/orders + repeat would memory-exhaust a Vercel function instance.",
      "recommendation": "Cycle 2: pair the order POST with the same per-session rate limiter recommended for the cart endpoints (e.g. 10 orders / minute). Belt-and-braces option: enforce a max-orders-per-day-per-retailer cap (e.g. 200) at createOrdersFromCart and return 429 when exceeded."
    }
  ],
  "promptInjectionRisk": "None. Neither handler issues an LLM call, performs LLM-mediated I/O, renders untrusted text, or executes shell or filesystem operations. The status query parameter is validated against a closed enum before reaching any logic; the POST has no body. All response data flows from typed in-memory rows through NextResponse.json's automatic escaping.",
  "decision": "GO",
  "requiredFixes": []
}
```